module 1

November 5, 2026

1 Module 1

In this assignment, you will work with ufo sightings data.

- The data includes various data points about individual ufo sightings
- Data File(s): ufo-sightings.csv

[4]:
```
############################################################
### EXECUTE THIS CELL BEFORE YOU TO TEST YOUR SOLUTIONS ###
############################################################

import imp, os, sys
sol = imp.load_compiled("solutions", "./solutions.py")
sol.get_solutions("ufo-sightings.csv")
from nose.tools import assert_equal
```

[11]:
```
'''
1. Import the csv module. Load and read the UFO sightings data set, from the␣
 ↪ufo-sightings.csv file,
into a DictReader inside a with statement.  Assume the data file is in the same␣
 ↪directory as the code.

Print the field names of the data set. Iterate over the reader to put the data␣
 ↪into a list name "ufosightings".

'''

import csv
filepath = "ufo-sightings.csv"
ufosightings = []

# your code here
with open(filepath, 'r', newline='', encoding='utf-8') as file:
    reader = csv.DictReader(file)
    for row in reader:
        ufosightings.append(row)

print(ufosightings[:2])
```

```
[OrderedDict([('datetime', '10/10/1949 20:30'), ('city', 'san marcos'),
('state', 'tx'), ('country', 'us'), ('shape', 'cylinder'), ('duration
(seconds)', '2700'), ('duration (hours/min)', '45 minutes'), ('comments', 'This
event took place in early fall around 1949-50. It occurred after a Boy Scout
meeting in the Baptist Church. The Baptist Church sit'), ('date posted',
'4/27/2004'), ('latitude', '29.8830556'), ('longitude ', '-97.9411111')]),
OrderedDict([('datetime', '10/10/1949 21:00'), ('city', 'lackland afb'),
('state', 'tx'), ('country', ''), ('shape', 'light'), ('duration (seconds)',
'7200'), ('duration (hours/min)', '1-2 hrs'), ('comments', '1949 Lackland
AFB&#44 TX.  Lights racing across the sky &amp; making 90 degree turns on a
dime.'), ('date posted', '12/16/2005'), ('latitude', '29.38421'), ('longitude ',
'-98.581082')])]
```

[8]:
```
############################
### TEST YOUR SOLUTION ###
############################

assert_equal(ufosightings, sol.ufosightings)
print("Success!")
```

```
Success!
```

[9]:
```
'''
2. How many sightings were there in total?  Put the count in␣
 ↪"ufosightings_count" and print the result.
'''
# your code here
ufosightings_count = len(ufosightings)
```

[10]:
```
############################
### TEST YOUR SOLUTION ###
############################

assert_equal(ufosightings_count, sol.ufosightings_count)
print("Success!")
```

```
Success!
```

[16]:
```
'''
3. How many sightings were there in the US?  Put the US sightings in␣
 ↪"sightings_us" and print.

Hint: Check for ufo sightings where the country is 'us'.

'''

# your code here
```

```
sightings_us = list(filter(lambda x:x['country'] == 'us', ufosightings))
sightings_us[:2]
```

[16]:
```
[OrderedDict([('datetime', '10/10/1949 20:30'),
              ('city', 'san marcos'),
              ('state', 'tx'),
              ('country', 'us'),
              ('shape', 'cylinder'),
              ('duration (seconds)', '2700'),
              ('duration (hours/min)', '45 minutes'),
              ('comments',
               'This event took place in early fall around 1949-50. It occurred
after a Boy Scout meeting in the Baptist Church. The Baptist Church sit'),
              ('date posted', '4/27/2004'),
              ('latitude', '29.8830556'),
              ('longitude ', '-97.9411111')]),
 OrderedDict([('datetime', '10/10/1956 21:00'),
              ('city', 'edna'),
              ('state', 'tx'),
              ('country', 'us'),
              ('shape', 'circle'),
              ('duration (seconds)', '20'),
              ('duration (hours/min)', '1/2 hour'),
              ('comments',
               'My older brother and twin sister were leaving the only Edna
theater at about 9 PM&#44…we had our bikes and I took a different route
home'),
              ('date posted', '1/17/2004'),
              ('latitude', '28.9783333'),
              ('longitude ', '-96.6458333')])]
```

[17]:
```
############################
### TEST YOUR SOLUTION ###
############################

assert_equal(sightings_us,  sol.sightings_us)
print("Success!")
```

```
Success!
```

[39]:
```
'''
4. Let's find the "fireball" sighting(s) that lasted more than ten seconds in␣
 ↪US.
Print the the datetime and state of each.  Put the data in "fball" and print␣
 ↪the result.

Note: Consider only the US sightings stored in "sightings_us".
```

```
- Cast the duration in seconds to a float (decimal).
- Check if duration is greater than 10.
- Check if the shape is "fireball".

'''

#First, define a Python function that checks if a given duration (seconds) is␣
 ↪"valid"
def is_valid_duration(duration_as_string):
# your code here
    #return duration_as_string.isnumeric() and float(duration_as_string) > 10
    try:
        return float(duration_as_string) > 10
    except ValueError:
        return False

fball = list(filter(lambda x: (x['shape'] == 'fireball') and␣
 ↪(is_valid_duration(x['duration (seconds)'])), sightings_us))
fball[:2]
```

[39]:
```
[OrderedDict([('datetime', '10/10/1968 19:00'),
              ('city', 'brevard'),
              ('state', 'nc'),
              ('country', 'us'),
              ('shape', 'fireball'),
              ('duration (seconds)', '180'),
              ('duration (hours/min)', '3 minutes'),
              ('comments',
               'silent red /orange mass of energy floated by three of us in
western North Carolina in the 60s'),
              ('date posted', '6/12/2008'),
              ('latitude', '35.2333333'),
              ('longitude ', '-82.7344444')]),
 OrderedDict([('datetime', '10/10/1998 03:30'),
              ('city', 'phoenix (north part)'),
              ('state', 'az'),
              ('country', 'us'),
              ('shape', 'fireball'),
              ('duration (seconds)', '15'),
              ('duration (hours/min)', '10 to 15 sec'),
              ('comments',
               'bright green moving north to north east. it was bright green and
kinda clear. quite fast too.'),
              ('date posted', '2/16/1999'),
              ('latitude', '33.4483333'),
              ('longitude ', '-112.0733333')])]
```

[40]:
```
#print(len(sol.fball), len(fball))
#for i in range(len(fball)):
#    if fball[i] != sol.fball[i]: print(i)
#print(fball[1778])
#print(sol.fball[1778])
```

[41]:
```
###########################
### TEST YOUR SOLUTION ###
###########################

assert_equal(fball, sol.fball)
print("Success!")
```

```
Success!
```

[42]:
```
'''
5. Sort the above list by duration. What was the datetime and duration of the␣
 ↪longest sighting?
Put the sorted list in "fballsorted" and print the result.

- Cast the duration in seconds to a float (decimal).
- Sort in reverse order.

'''

# your code here
fballsorted = sorted(fball, key=lambda x: -float(x['duration (seconds)']))
fballsorted[0]
```

[42]:
```
OrderedDict([('datetime', '8/3/2008 21:00'),
             ('city', 'virginia beach'),
             ('state', 'va'),
             ('country', 'us'),
             ('shape', 'fireball'),
             ('duration (seconds)', '10526400'),
             ('duration (hours/min)', '4 months'),
             ('comments',
              'this object was very high up and emmited no sound.&#33'),
             ('date posted', '8/12/2008'),
             ('latitude', '36.8527778'),
             ('longitude ', '-75.9783333')])
```

[43]:
```
###########################
### TEST YOUR SOLUTION ###
###########################

assert_equal(fballsorted, sol.fballsorted)
```

```
print("Success!")
```

```
Success!
```

[44]:
```
'''
6. What state had the longest lasting "fireball"?  Put the state in "state" and print the result.

- Check if the shape is "fireball".
- Cast the duration in seconds to a float (decimal).
- Get the record with the largest (max) duration in seconds.
- Get the state for the record.

'''

# your code here
state = fballsorted[0]['state']
state
```

[44]:
```
'va'
```

[45]:
```
############################
### TEST YOUR SOLUTION ###
############################

assert_equal(state, sol.state)
print("Success!")
```

```
Success!
```

[53]:
```
'''
7. Let's assume that any sighting (of any shape) of 0 seconds is insignificant. Write code to filter out these extraneous records and get the shortest sighting overall now.
Put the minimum duration in "min_duration" and print the result.
Use ufosightings
Note: Consider all sightings stored in "ufosightings".

'''

# your code here

def is_positive_duration(duration_as_string):
# your code here
    #return duration_as_string.isnumeric() and float(duration_as_string) > 10
    try:
        return float(duration_as_string) > 0
```

```
    except ValueError:
        return False


significant_sightings_us = list(filter(lambda x:
 ↪is_positive_duration(x['duration (seconds)']), ufosightings))

significant_sightings_us = sorted(significant_sightings_us, key=lambda x:␣
 ↪float(x['duration (seconds)']))
min_duration = float(significant_sightings_us[0]['duration (seconds)'])
```

[54]:
```
############################
### TEST YOUR SOLUTION ###
############################

assert_equal(min_duration, sol.min_duration)
print("Success!")
```

Success!

[72]:
```
'''
8. What are the top 3 shapes sighted, and how many sightings were there for␣
 ↪each?

Note: Consider all sightings stored in "ufosightings".

- Create a new list "sightings_shapes" containing values from the "shape"␣
 ↪column in ufosightings.
- Create a new dictionary "count" with values of that column as keys and the␣
 ↪counts as values.
- Get a list of the dictionary keys and values using the items() method.  This␣
 ↪will return a list of key:value pairs.
Sort the list of key:value pairs in reverse order, from greatest (most␣
 ↪sightings) to least.

Get the top 3 and store in "top3shapes".  Print the result.

'''


#Create a new list containing values from the "shape" column in ufosightings.
# your code here
sightings_shapes = [x['shape'] for x in ufosightings]

#Create a new dictionary with values of that column as keys and the counts as␣
 ↪values.
# your code here
```

```
count = {}
for shape in sightings_shapes:
    count[shape] = count.get(shape, 0) + 1

#Get a list of the dictionary keys and values (use the items() method) and sort␣
 ↪them in reverse order, from greatest (most sightings) to least.
#Get and print the top 3.
# your code here
#sorted_shapes = sorted(count, key=count.get, reverse=True)
sorted_shapes = sorted(count.items(), key=lambda x: x[1], reverse=True)
#sorted_shapes
top3shapes = sorted_shapes[:3] #[x[0] for x in sorted_shapes[:3]]
top3shapes
```

[72]:
```
[('light', 16565), ('triangle', 7865), ('circle', 7608)]
```

[73]:
```
############################
### TEST YOUR SOLUTION ###
############################

assert_equal(sightings_shapes, sol.sightings_shapes)
print("Success!")
```

```
Success!
```

[74]:
```
############################
### TEST YOUR SOLUTION ###
############################

assert_equal(count, sol.count)
print("Success!")
```

```
Success!
```

[75]:
```
############################
### TEST YOUR SOLUTION ###
############################

assert_equal(top3shapes, sol.top3shapes)
print("Success!")
```

```
Success!
```

[ ]:
```

```